

PROFILE

React·TypeScript 기반 프론트엔드 개발을 중심으로 WebView 제품, 데이터 분석 UI, BFF와 사내 AI Agent 런타임을 개발해왔습니다. 성능 문제는 측정 결과를 바탕으로 개선하고, 복잡한 요구사항은 데이터 흐름과 각 모듈의 역할을 먼저 정리합니다. 제품 개발에 필요할 때는 NestJS·FastAPI 등 백엔드 영역도 함께 다룹니다.

EXPERIENCE

2024.09 - 현재

Frontend Developer

2년

파이레테크놀로지

Frontend Developer

WebView 거래 제품과 BFF, 블록 스캐너 성능 개선, 사내 AI Agent 런타임 개발을 맡고 있습니다.

TypeScript

React

Next.js

TailwindCSS

TanStack Query

Jotai

NestJS

wagmi · viem

Chakra UI

Vitest

Playwright

Sentry

- 블록 스캐너 주요 목록에 단계적 렌더링과 WebSocket 배치를 적용해 로컬 Lighthouse 기준 LCP 평균 **45%**, TBT 평균 **74%** 단축
- Explorer를 Next.js Pages Router에서 App Router로 전환하고 React·Chakra UI 마이그레이션과 SSR hydration 오류 수정
- Pockie의 BiFi·Swap·BTCFi WebView 거래 화면과 NestJS 기반 Swap BFF 개발 — 견적·거래 데이터 생성·상태 조회 API 연동
- BTCFi Partners의 파트너별 거래 흐름을 공통 모듈에서 분리하고, 공유 UI·데이터 계층의 경계를 테스트로 검증
- 사내 AI Agent의 Claude SDK 실행 경로를 Codex App Server로 전환하고, 도구별 실행 권한과 자동 코드 리뷰 검증 흐름 개선

2021.07 - 2024.09

3년 2개월

Full-stack Engineer

→ Frontend Developer

→ Frontend Part Leader

스마트마인드

Frontend Developer → Frontend Part Leader

AI 데이터 분석 플랫폼 Workspace의 SQL Editor·Query Viewer 모듈을 개발했으며, pnpm·Turborepo 모노레포 전환과 Microfrontend 설계를 주도했습니다. 후반에는 프론트엔드 파트 리더로 팀 운영을 담당했습니다.

TypeScript

React

Next.js

Vite

Monaco Editor

ANTLR

pnpm · Turborepo

Playwright

FastAPI

Docker Compose

- pnpm·Turborepo 모노레포와 빌드 캐시를 도입해 당시 측정 기준 앱별 빌드 시간을 **7분 → 1분**으로 단축
- 번들 분리·dynamic import로 Workspace 초기 로드 범위를 줄여 당시 측정 기준 First Load 최대 **40%** 단축
- AI 팀이 정의한 ANTLR 문법을 Monaco Editor에 연결해 SQL 구문 강조·오류 진단·키워드 자동완성 구현
- Microfrontend 설계·구현 — iframe+postMessage(Lab·Main 간 통신), module federation(Query Manager·File Manager)
- 비정형 데이터 Query Viewer 성능 개선 — 서버 페이지네이션·가상 스크롤·미디어 lazy loading으로 대용량 결과 렌더링 안정화
- FastAPI·Docker Compose로 AI 예측 모델 서빙 환경과 데이터 수집·전처리 파이프라인 개발
- 2023년부터 본인 포함 2~3명 프론트엔드 파트의 스크럼 운영·업무 배분·우선순위 결정 담당

KEY ACHIEVEMENTS

01 /

블록 스캐너(Explorer) 성능 최적화

Route	LCP (before → after)	TBT (before → after)	개선
/blocks	3.7s 1.9s	1,010ms 270ms	LCP ↓49% · TBT ↓73%
/txs	3.7s 2.2s	1,330ms 200ms	LCP ↓41% · TBT ↓85%
/tokens	3.3s 1.8s	510ms 190ms	LCP ↓45% · TBT ↓63%

- 최대 75개 row를 노출하는 테이블에서 가상화 적용 시 레이아웃 공백 문제가 있어 롤백하고, 초기 row 수를 제한한 단계적 렌더링으로 전환
- WebSocket 30초 배치 처리로 실시간 업데이트에 따른 반복 렌더링 축소
- Jest를 Vitest로 전환하고 Playwright로 단계적 렌더링·레이아웃 시프트·WebSocket 갱신 회귀 테스트 추가

02 /

WebView 거래 제품·파트너별 코드 구조

- Pockie miniDApp — 모바일 앱·Chrome Extension에서 사용하는 BiFi-Swap·BTCFi 거래 화면 개발 및 postMessage 기반 지갑 기능 연동
- Pockie miniDApp — iOS·Android·Extension과 앱 버전별 feature flag를 BFF config v2에 구현해 클라이언트 재배포 없이 기능 노출 제어
- BTCFi Partners — 파트너별 예치·출금·클레임 흐름은 별도 모듈로 분리하고 UI·데이터 계층은 공유하도록 정리, 모듈 간 의존 규칙을 CI 테스트에 추가

03 /

사내 AI Agent 런타임 개선

- Claude SDK 실행 경로를 Codex App Server 세션·worker로 전환하고 요청별 상태·취소 처리 분리
- 세션 시작 시 모든 MCP 서버를 실행하던 방식을 필요한 서버만 호출 시 실행하는 lazy proxy로 변경
- 요청 전체의 읽기·쓰기 분류 대신 도구별 외부 영향·되돌릴 수 있는지를 기준으로 자동 실행·승인·차단을 판단하도록 권한 정책 재설계
- 자동 코드 리뷰에 저장소별 규칙·confidence·CI 상태 검증을 적용하고, 최종 merge는 사람이 확인하도록 운영

04 /

DApp 데이터 조회 구조 최적화

- Gas Top-up — SDK의 중복 자산 조회와 불필요한 vault 조회를 제거하고 갱신 주기를 조정해 최초 API 호출을 25~46회~17회, 주기 호출 분당 25회~1~19회로 축소
- BiFi WebView — bridge pair 최초 1회+pair별 N회 조회를 outbound·inbound 각 1회로 재구성하고 React Query 캐시로 조회 결과 재사용

CERTIFICATIONS & ACTIVITIES

AWS Certified Solutions Architect – Associate (2026)
정보처리기사 (2019)

OSSCA — githru-vscode-ext (2023)
SmileGate Membership AI 1기 (2021)
OSSCA — TS Handbook 한글화 (2020)
AUSG (AWSKRUG University Student Group) 1기 (2019)

LANGUAGES

영어 — 기초 비즈니스 (Opic IM1)
일본어 — 기초 비즈니스 (JLPT N3)